Conceptos básicos de CUDA

Cómo pensar un programa para la GPU

Universidad Industrial de Santander
Escuela de Ingeniería de Sistemas e Informática

Agenda

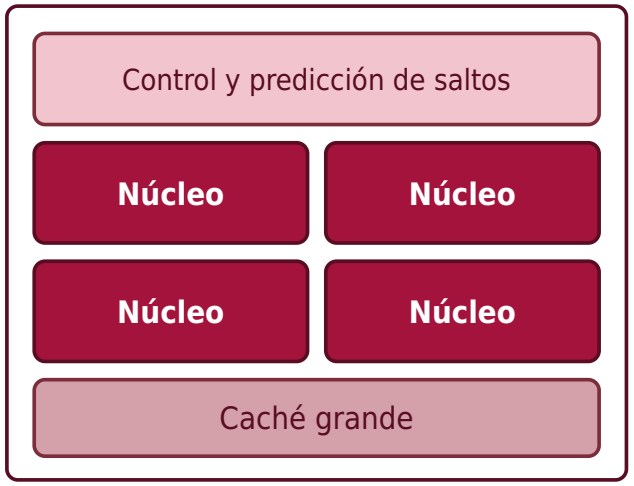
1. **¿Por qué una GPU?**
2. **Host y device**: dos computadores en uno
3. **Kernels** y la jerarquía de hilos
4. **El índice global**: a cada hilo, su dato
5. **Warps y memoria**: lo que conviene saber desde el principio
6. Síntesis

1. ¿Por qué una GPU?

Muchos núcleos sencillos en lugar de pocos núcleos complejos

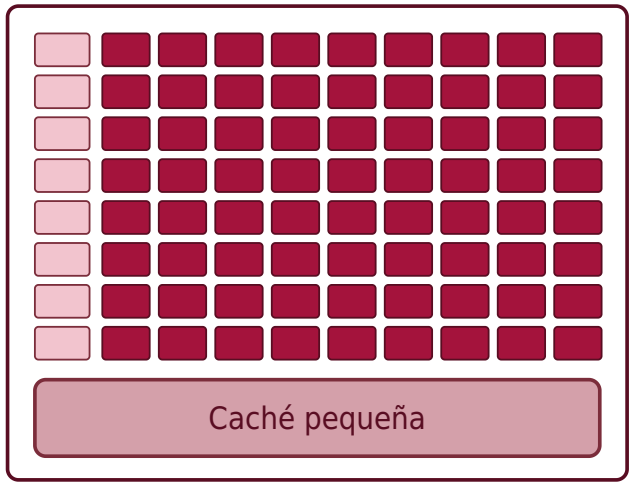
CPU y GPU: dos filosofías

CPU: pocos núcleos potentes



Baja latencia en tareas secuenciales

GPU: miles de núcleos sencillos



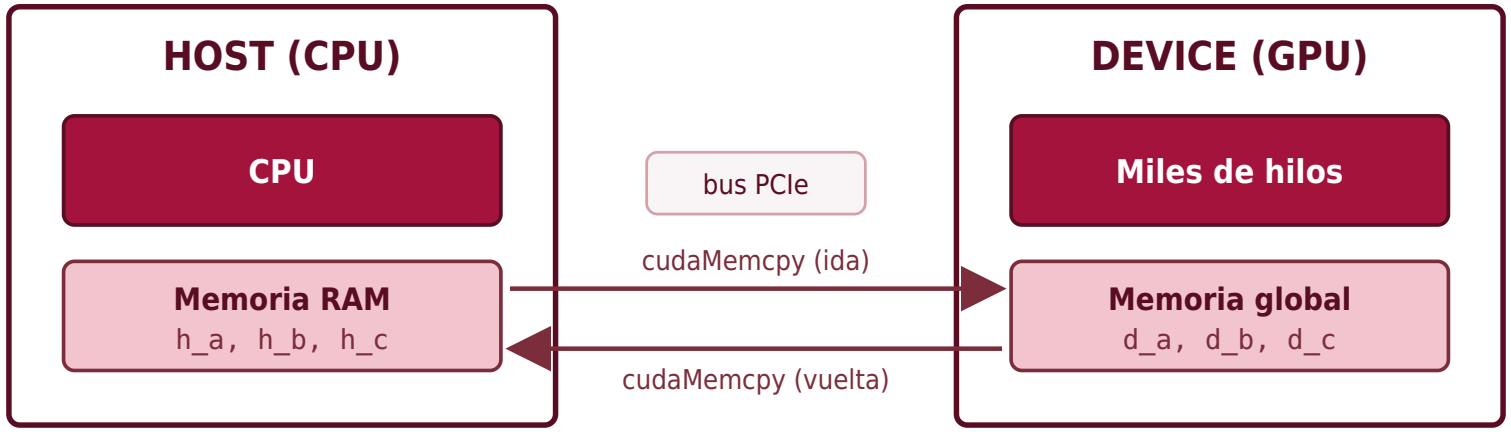
Alto rendimiento en datos paralelos

> La GPU gana cuando hay **la misma operación sobre muchísimos datos** independientes.

2. Host y device

La CPU dirige; la GPU calcula

Dos procesadores, dos memorias



Son memorias separadas: los datos deben copiarse explícitamente

- **Host**: la CPU y su memoria RAM. Allí empieza y termina el programa.
- **Device**: la GPU y su memoria propia.
- Convención útil: prefijo `h_` para datos del host y `d_` para datos del device.

El flujo de todo programa CUDA



Solo el paso 3 se ejecuta en la GPU; los demás los ordena la CPU

```
cudaMalloc(&d_x, bytes);                                  // 1
cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice);      // 2
mi_kernel<<<bloques, hilos>>>(d_x, n);                    // 3
cudaMemcpy(h_x, d_x, bytes, cudaMemcpyDeviceToHost);      // 4
cudaFree(d_x);                                            // 5
```

3. Kernels e hilos

Una función que se ejecuta miles de veces a la vez

¿Qué es un kernel?

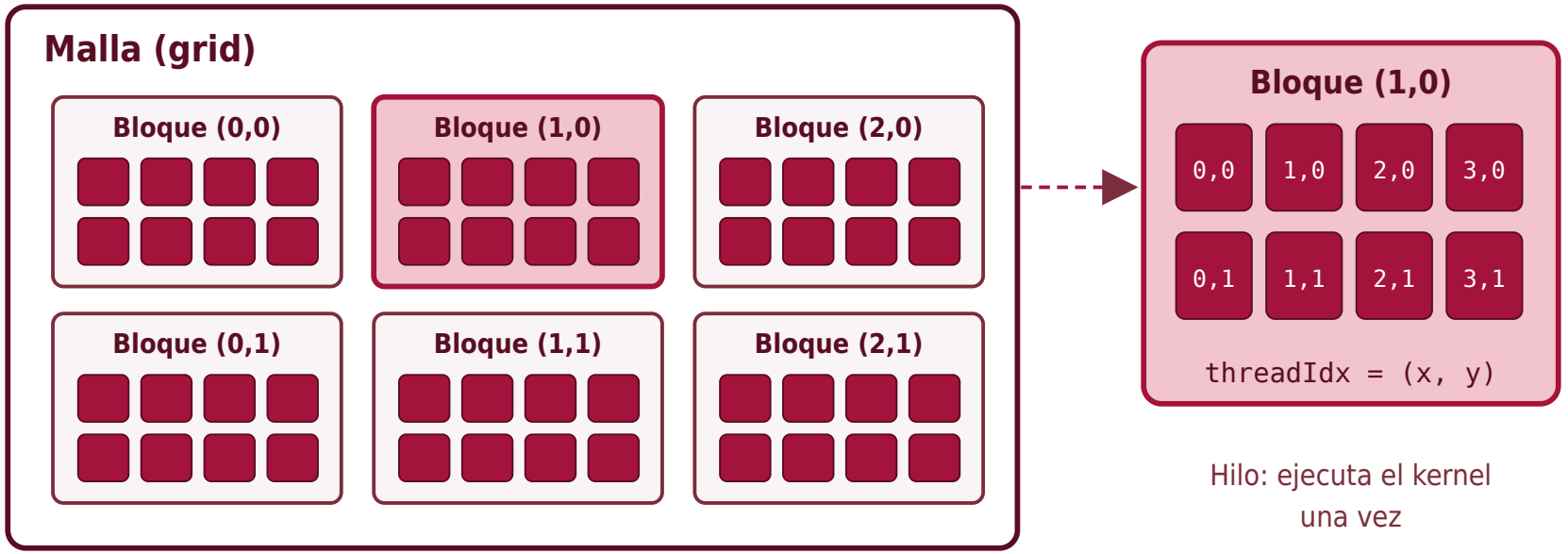
Una función marcada con `__global__`: se **llama desde la CPU** y se **ejecuta en la GPU**.

```
__global__ void duplicar(float *x, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // ¿quién soy?
    if (i < n)                                       // ¿me toca trabajar?
        x[i] = 2.0f * x[i];                          // mi parte del trabajo
}
```

Lanzamiento: `duplicar<<<bloques, hilos_por_bloque>>>(d_x, n);`

> No hay ciclo `for`: **cada hilo procesa un elemento**. El ciclo lo "reemplaza" la cantidad de hilos.

La jerarquía: malla → bloques → hilos



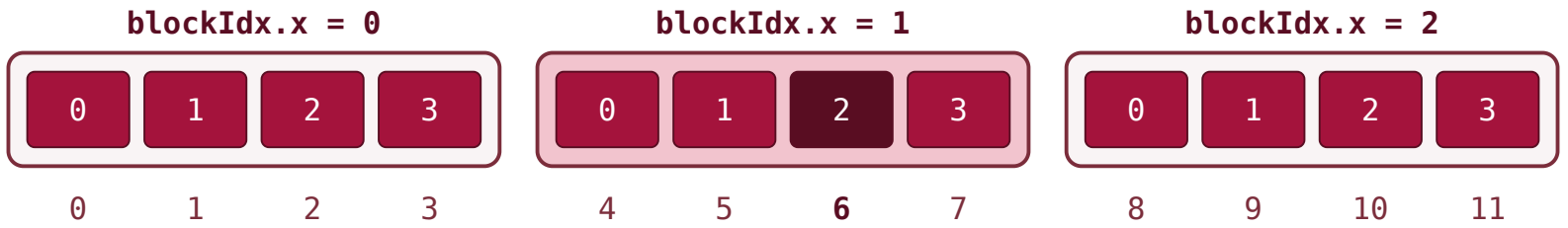
Hilo: ejecuta el kernel una vez

- Los hilos se agrupan en **bloques** (hasta 1024 hilos por bloque).
- Los bloques forman una **malla** (*grid*).
- Cada hilo conoce su posición: `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.

4. El índice global

A cada hilo, su dato

¿Qué elemento le toca a cada hilo?



Dentro de la caja: threadIdx.x · Debajo: índice global i en el arreglo

$$i = blockIdx.x \cdot blockDim.x + threadIdx.x$$

Ejemplo resaltado: i = 1 · 4 + 2 = 6

¿Cuántos bloques lanzar?

Si `n = 1000` y usamos 256 hilos por bloque:

$$\text{bloques} = \left\lceil \frac{1000}{256} \right\rceil = 4 \quad \Rightarrow \quad 4 \times 256 = 1024 \text{ hilos}$$

```
int hilos   = 256;
int bloques = (n + hilos - 1) / hilos;   // división entera redondeando hacia arriba
```

Sobran 24 hilos: por eso el kernel lleva `if (i < n)`. **Sin esa condición, se escribe fuera del arreglo.**

En dos dimensiones: matrices

```
dim3 hilos(16, 16);                                    // 256 hilos por bloque
dim3 bloques((N + 15) / 16, (N + 15) / 16);
matmul<<<bloques, hilos>>>(d_A, d_B, d_C, N);
```

Dentro del kernel:

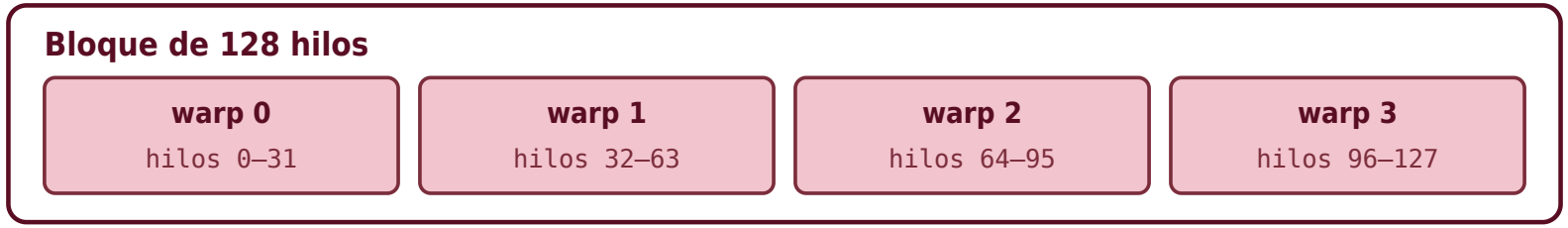
```
int fila = blockIdx.y * blockDim.y + threadIdx.y;
int col  = blockIdx.x * blockDim.x + threadIdx.x;
if (fila < N && col < N) { /* calcular C[fila][col] */ }
```

> Es la misma idea, repetida en cada eje: **un hilo por elemento de la matriz**.

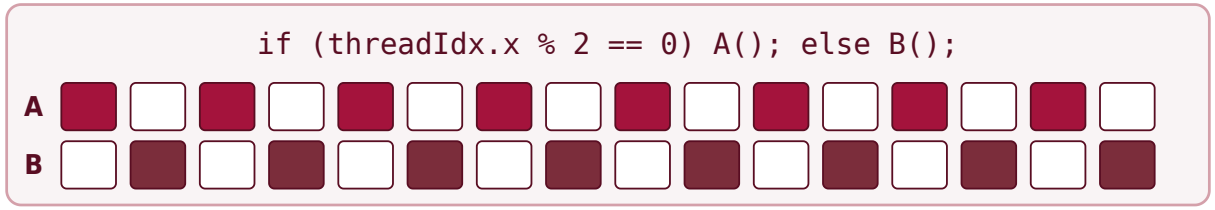
5. Warps y memoria

Dos detalles que afectan el rendimiento

Warps: los hilos avanzan en grupos de 32

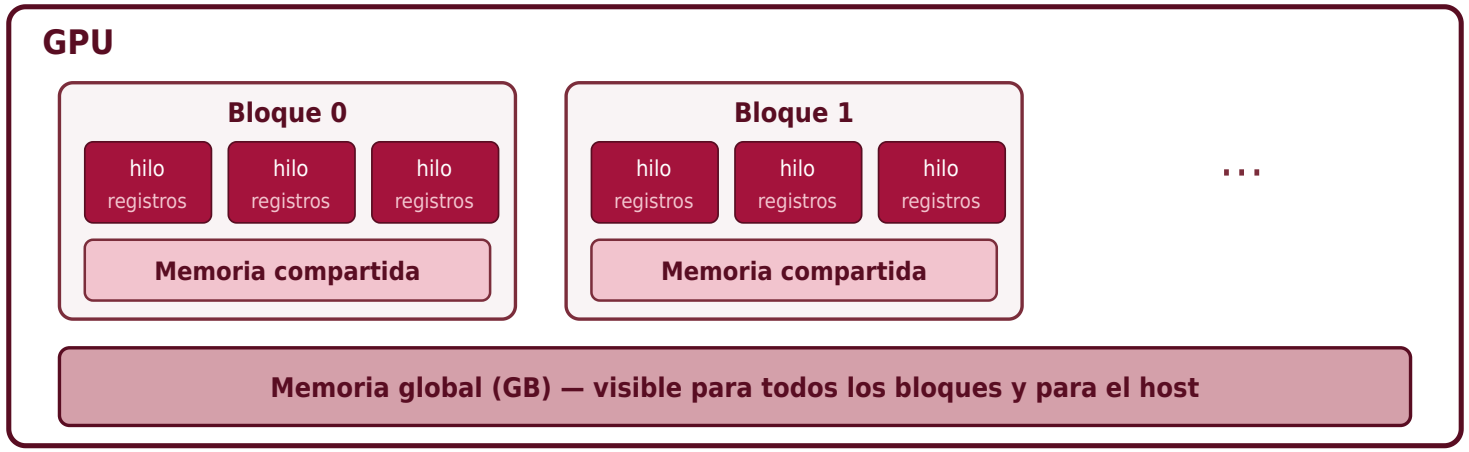


Un warp = 32 hilos que ejecutan la misma instrucción a la vez (SIMT)



Divergencia: las dos ramas se ejecutan una después de la otra

Jerarquía de memoria de la GPU



Registros	privados de cada hilo	**máxima velocidad**
Compartida	del bloque (KB)	**muy rápida**
Global	de toda la GPU (GB)	**lenta**

más lenta

El mismo kernel en Python (Numba)

```python
from numba import cuda

@cuda.jit
def duplicar(x):
    i = cuda.grid(1)          # = blockIdx.x * blockDim.x + threadIdx.x
    if i < x.size:
        x[i] = 2.0 * x[i]

d_x = cuda.to_device(h_x)                    # copiar host → device
duplicar[bloques, hilos](d_x)                # lanzar
h_x = d_x.copy_to_host()                     # copiar device → host
```

Cambia la sintaxis, **no cambian los conceptos**: índice global, condición de borde, copias.

Errores frecuentes al empezar

Error	Consecuencia
Olvidar `if (i < n)`	Escritura fuera del arreglo
Usar un puntero del host dentro del kernel	Fallo o resultados basura
Copiar datos en cada paso de un ciclo	El programa es más lento que en CPU
Más de 1024 hilos por bloque	El kernel no se lanza (sin detener el programa)
No revisar errores	Fallos silenciosos

```
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) printf("Error: %s\n", cudaGetErrorString(err));
```

Síntesis

1. La GPU sirve para aplicar **la misma operación a muchos datos**.
2. **Host y device tienen memorias separadas**: hay que copiar.
3. Un **kernel** se ejecuta una vez por hilo; los hilos se organizan en **bloques** y **mallas**.
4. `i = blockIdx.x * blockDim.x + threadIdx.x` y **siempre** `if (i < n)`.
5. Los hilos avanzan en **warps de 32**; la memoria tiene **niveles** con velocidades muy distintas.